

Dans le contexte actuel de transformation numérique du secteur bancaire, les systèmes automatisés de scoring de crédit jouent un rôle central dans l'évaluation des risques et la prise de décision. L'enjeu n'est plus seulement de prédire si un client remboursera ou non un crédit, mais de rendre ces prédictions **utilisables, fiables et optimisées** pour un usage opérationnel. Deux leviers majeurs permettent d'améliorer l'efficacité de ces systèmes :

(1) l'optimisation des seuils de décision, qui permet d'ajuster la frontière entre les décisions d'acceptation ou de rejet d'un dossier;

(2) la calibration des probabilités prédictives, qui vise à garantir que les scores produits par le modèle reflètent fidèlement les risques réels. L'objectif de ce projet est ainsi double : d'une part, améliorer la qualité des décisions en tenant compte des contraintes métiers (taux d'acceptation, coût d'erreur, équilibre entre précision et rappel); et d'autre part, garantir que les **probabilités générées par les modèles soient bien calibrées, c'est-à-dire qu'un score de 0,8 corresponde réellement à 80. Pour cela, le travail est structuré en trois phases principales :

— Phase 1 – Analyse des seuils: optimisation mono-objectif (F1-score, coût, précision/rappel) et multi-objectifs (front de Pareto, compromis performance/stabilité);

— Phase 2 – Calibration des probabilités : évaluation de la calibration (Brier Score, test HL) et application de méthodes de recalibration (Platt scaling, isotonic regression, binning);

— Phase 3 – Stratégies de décision: élaboration de règles complexes par segments, intégration de contraintes réglementaires, et simulation d'impact économique. Ce document présente les approches mises en œuvre, les résultats obtenus, ainsi que les recommandations stratégiques pour un déploiement fiable et performant d'un système de scoring calibré et optimisé.

Préparation des données pour le modèle de scoring

Dans ce bloc, nous préparons notre jeu de données pour l'entraînement d'un modèle de scoring. Voici les principales étapes effectuées :

Chargement du fichier : nous importons le fichier contenant les demandes de crédit.

Nettoyage : certaines colonnes non pertinentes pour le scoring (identifiants, date, genre) sont supprimées car elles peuvent introduire du bruit ou des biais.

Encodage des variables catégorielles : nous transformons les variables de type texte en valeurs numériques à l'aide du `LabelEncoder`, pour permettre leur traitement par les algorithmes de machine learning.

Séparation des variables :

- `X` contient les variables explicatives (caractéristiques du client),
 - `y` contient la variable cible (`Defaut_12M`), qui indique si un client a fait défaut dans les 12 mois.
- Division du jeu de données** : nous séparons les données en un ensemble d'entraînement (`70%`) et un ensemble de test (`30%`), en conservant la

proportion des classes (`stratify=y`). Le paramètre `random_state=42` est fixé pour garantir la reproductibilité des résultats.

```
In [3]: import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_recall_curve, f1_score, confusion_matrix,

# Charger Le fichier (à adapter selon ton nom de fichier)
df = pd.read_csv("dataset_scoring_complet.csv")

# Supprimer Les colonnes non pertinentes
df = df.drop(columns=['ID_Client', 'Date_Demande', 'Genre'])

# Encodage LabelEncoder (rapide) sur Les colonnes catégorielles
for col in df.select_dtypes(include='object').columns:
    df[col] = LabelEncoder().fit_transform(df[col])

# Définir X (features) et y (cible)
X = df.drop(columns=['Defaut_12M'])
y = df['Defaut_12M']

# Séparation en train / test (30% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, test_size=
```

```
In [ ]:
```

```
In [4]: # Vérifie Les NaN
print(X_train.isnull().sum())

# Vérifie Les infinis
print(np.isinf(X_train).sum())
```

```

Age                                0
Situation_Familiale                0
Nb_Enfants                         0
Situation_Professionnelle          0
Anciennete_Emploi                  104
Revenus                            170
Montant_Demande                    0
Score_Bureau                       67
Ratio_Dette_Revenu                 0
Nb_Comptes                         0
Anciennete_Banque                  0
Incidents_Bancaires                0
Temps_Demande_Min                  0
Nb_Modifications                   0
Canal_Demande                      0
Heure_Demande                      0
Ratio_Demande_Revenu               0
Age_Groupe                         0
Revenu_Groupe                      0
dtype: int64
Age                                0
Situation_Familiale                0
Nb_Enfants                         0
Situation_Professionnelle          0
Anciennete_Emploi                  0
Revenus                            0
Montant_Demande                    0
Score_Bureau                       0
Ratio_Dette_Revenu                 0
Nb_Comptes                         0
Anciennete_Banque                  0
Incidents_Bancaires                0
Temps_Demande_Min                  0
Nb_Modifications                   0
Canal_Demande                      0
Heure_Demande                      0
Ratio_Demande_Revenu               0
Age_Groupe                         0
Revenu_Groupe                      0
dtype: int64

```

Imputation des valeurs manquantes

Ce bloc traite les **valeurs manquantes** dans les données, ce qui est une étape essentielle avant d'entraîner un modèle. En effet, la plupart des algorithmes de machine learning (dont la régression logistique) ne peuvent pas gérer directement des valeurs manquantes.

- Pour les **variables numériques**, on applique une stratégie d'imputation par la **moyenne** (`strategy='mean'`). Cela permet de remplacer les valeurs manquantes par la moyenne de la variable, ce qui évite de perdre de l'information.
- Le résultat est reconverti en `DataFrame` avec les mêmes noms de colonnes qu'à l'origine.
- Pour les variables **catégorielles** (si présentes), une stratégie alternative par la **modalité la plus fréquente** (`most_frequent`) est commentée mais prête à être utilisée si nécessaire.

```
In [5]: # Pour Les variables numériques
imputer = SimpleImputer(strategy='mean') # ou 'median'
X_train_imputed = pd.DataFrame(imputer.fit_transform(X_train), columns=X_train.columns)
X_test_imputed = pd.DataFrame(imputer.fit_transform(X_test), columns=X_test.columns)
# Pour Les variables catégorielles
# imputer_cat = SimpleImputer(strategy='most_frequent')
```

Entraînement du modèle de régression logistique

Dans ce bloc, nous entraînons un **modèle de régression logistique** sur les données d'entraînement préparées (`X_train_imputed`).

- La **régression logistique** est un modèle de classification binaire adapté ici pour prédire la probabilité qu'un client fasse défaut (`Defaut_12M`).
- Le paramètre `max_iter=1000` est utilisé pour s'assurer que l'optimisation converge, même si le modèle met plus de temps à s'ajuster.
- Le modèle est ensuite ajusté (`fit`) aux données d'entraînement, ce qui lui permet d'apprendre les relations entre les caractéristiques des clients et leur comportement de remboursement.

```
In [6]: #entraîner le modèle
model = LogisticRegression(max_iter=1000)
model.fit(X_train_imputed, y_train)
```

```
Out[6]: LogisticRegression(max_iter=1000)
```

Prédiction des probabilités de défaut sur les données de test

Ce bloc permet d'obtenir les **probabilités prédites** par le modèle pour chaque individu du jeu de test.

- La méthode `predict_proba()` retourne, pour chaque observation, la probabilité d'appartenir à chaque classe (0 = bon payeur, 1 = mauvais payeur).
- Ici, nous sélectionnons uniquement la **probabilité associée à la classe 1** (`[:, 1]`), c'est-à-dire la **probabilité que le client fasse défaut** dans les 12 mois.
- L'affichage des 50 premières probabilités permet d'inspecter la sortie du modèle et de valider que les valeurs sont bien comprises entre 0 et 1.

```
In [7]: # Prédire les probabilités sur le test set
y_prob = model.predict_proba(X_test_imputed)[:, 1]
print(y_prob[:50])
```

```
[0.58587495 0.2033721 0.10460214 0.16341441 0.15902784 0.1159185
0.02891354 0.07469499 0.06394308 0.16490724 0.36529484 0.37658003
0.29629766 0.08929989 0.14677304 0.21815046 0.03700648 0.09994654
0.14810202 0.2998728 0.16119336 0.04790084 0.24208351 0.61581159
0.27022715 0.04496957 0.14064268 0.22987221 0.20647578 0.13410376
0.2800684 0.15633876 0.18930108 0.57361885 0.08085406 0.13288053
0.26255801 0.06611338 0.06746381 0.00997606 0.18295809 0.09802304
0.16224915 0.02736743 0.48255533 0.16685246 0.11303461 0.21418373
0.03464585 0.1311046 ]
```

Interprétation des coefficients du modèle

Ce bloc permet d'analyser l'importance et le **sens d'influence de chaque variable** dans la régression logistique.

- Chaque **coefficient** représente l'impact (positif ou négatif) de la variable correspondante sur la **probabilité de défaut**.
 - Un coefficient **positif** signifie que plus la variable augmente, plus la probabilité de défaut augmente.
 - Un coefficient **négatif** signifie que plus la variable augmente, plus la probabilité de défaut diminue.
- Les coefficients sont extraits avec `model.coef_`, associés aux noms des variables, puis triés par ordre décroissant pour identifier les **variables les plus influentes** sur le risque de défaut.

```
In [8]: # Afficher les coefficients pour interpréter les variables
coefficients = pd.DataFrame({
    'Variable': X_train.columns,
    'Coefficient': model.coef_[0]
}).sort_values(by='Coefficient', ascending=False)

print(coefficients)
```

	Variable	Coefficient
16	Ratio_Demande_Revenu	0.144674
3	Situation_Professionnelle	0.117194
9	Nb_Comptes	0.092555
13	Nb_Modifications	0.057896
8	Ratio_Dette_Revenu	0.047651
11	Incidents_Bancaires	0.042261
0	Age	0.039245
18	Revenu_Groupe	0.026920
14	Canal_Demande	0.022827
15	Heure_Demande	0.021220
1	Situation_Familiale	0.009717
12	Temps_Demande_Min	0.002434
5	Revenus	0.000116
4	Anciennete_Emploi	0.000000
6	Montant_Demande	-0.000038
2	Nb_Enfants	-0.002916
7	Score_Bureau	-0.007504
17	Age_Groupe	-0.021745
10	Anciennete_Banque	-0.028862

Évaluation de la calibration du modèle (avant recalibrage)

Ce bloc permet de visualiser la **calibration** du modèle de régression logistique, c'est-à-dire la cohérence entre les **probabilités prédites** et la **réalité observée**.

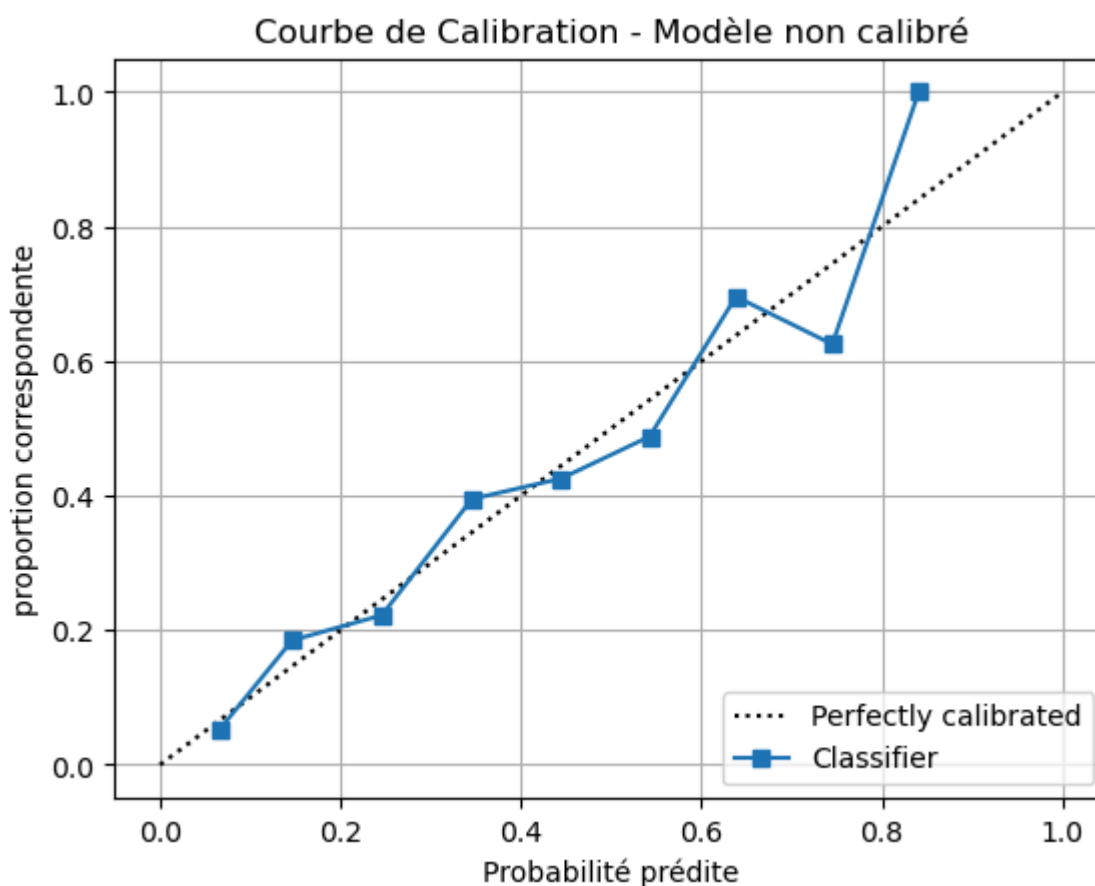
- La **courbe de calibration** (ou diagramme de fiabilité) compare, pour différentes tranches de probabilité, la proportion de clients ayant effectivement fait défaut.
- Une **ligne parfaitement diagonale** représenterait un modèle bien calibré : par exemple, parmi les clients ayant une probabilité prédite de 0.3, 30 % font réellement défaut.
- Ici, on utilise `CalibrationDisplay.from_predictions` avec 10 groupes (`n_bins=10`) pour tracer cette relation sur le jeu de test avant recalibration.

- Cette évaluation permet de détecter une éventuelle **surconfiance** ou **sous-confiance** du modèle dans ses prédictions.

```
In [9]: #Évaluer la calibration actuelle (avant recalibration)
from sklearn.calibration import CalibrationDisplay, calibration_curve
import matplotlib.pyplot as plt

# Prédiction de probabilité sur le test
y_prob = model.predict_proba(X_test_imputed)[: , 1]

# Affichage de la courbe de calibration
display = CalibrationDisplay.from_predictions(y_test, y_prob, n_bins=10)
plt.title("Courbe de Calibration - Modèle non calibré")
plt.xlabel("Probabilité prédite")
plt.ylabel("proportion correspondante")
plt.grid()
plt.show()
```



Évaluation avancée de la calibration du modèle

Ce bloc évalue la **qualité de la calibration** du modèle selon plusieurs méthodes complémentaires, et pose les bases pour comparer plusieurs jeux de probabilités (modèle brut, calibré par Platt, calibré par isotonic).

1. Diagramme de fiabilité personnalisé La fonction `plot_reliability_diagram()` trace plusieurs **diagrammes de calibration** sur le même graphique, permettant de comparer différents modèles ou techniques de recalibrage. La diagonale représente une calibration parfaite.

2. Test d'Hosmer-Lemeshow La fonction `hosmer_lemeshow()` met en œuvre un test statistique évaluant si les probabilités prédites sont significativement différentes des taux de défaut observés.

- Une **p-value élevée (> 0.05)** indique que le modèle est bien calibré.
- Une **p-value faible** indique un écart significatif entre prédictions et observations.

3. Brier Score Le **Brier Score** mesure l'écart quadratique moyen entre la probabilité prédite et la réalité (0 ou 1). Plus il est **proche de 0**, meilleure est la calibration.

4. Indice de sur/sous-confiance La fonction `indice_surconfiance()` mesure si le modèle est en moyenne **trop confiant** (surévalue les probabilités) ou **pas assez confiant** :

- Un indice **positif** indique une **surconfiance** (le modèle surestime les risques).
- Un indice **négatif** indique une **sous-confiance**.

Enfin, on applique ces méthodes à un (ou plusieurs) jeu(x) de probabilités

(`y_probs_list`) : celles du modèle brut (`y_prob`), ou d'autres modèles recalibrés (`y_prob_platt` , `y_prob_iso`) si ajoutés.

```
In [10]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.calibration import calibration_curve
from sklearn.metrics import brier_score_loss
from scipy.stats import chi2

# Fonction pour Le diagramme de fiabilité
def plot_reliability_diagram(y_true, prob_sets, names):
    plt.figure(figsize=(7, 4))
    for probs, label in zip(prob_sets, names):
        prob_true, prob_pred = calibration_curve(y_true, probs, n_bins=10, strat
        plt.plot(prob_pred, prob_true, marker='o', label=label)
    plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Parfaitement c
    plt.xlabel('Probabilité prédite')
    plt.ylabel('Fréquence observée')
    plt.title('Diagrammes de fiabilité')
    plt.legend()
    plt.grid()
    plt.show()

# Fonction pour Le test d'Hosmer-Lemeshow
def hosmer_lemeshow(y_true, y_prob, g=10):
    data = pd.DataFrame({'y': y_true, 'prob': y_prob})
    data['bucket'] = pd.qcut(data['prob'], q=g, duplicates='drop')
    obs = data.groupby('bucket')['y'].sum()
    exp = data.groupby('bucket')['prob'].sum()
    n = data.groupby('bucket')['y'].count()
    hl_stat = ((obs - exp)**2 / (exp * (1 - exp / n))).sum()
    df = g - 2
    p_value = 1 - chi2.cdf(hl_stat, df)
    return hl_stat, p_value

# Fonction pour L'indice de sur/sous-confiance
def indice_surconfiance(y_true, y_prob):
```

```

prob_true, prob_pred = calibration_curve(y_true, y_prob, n_bins=10, strategy
return np.mean(prob_pred - prob_true)

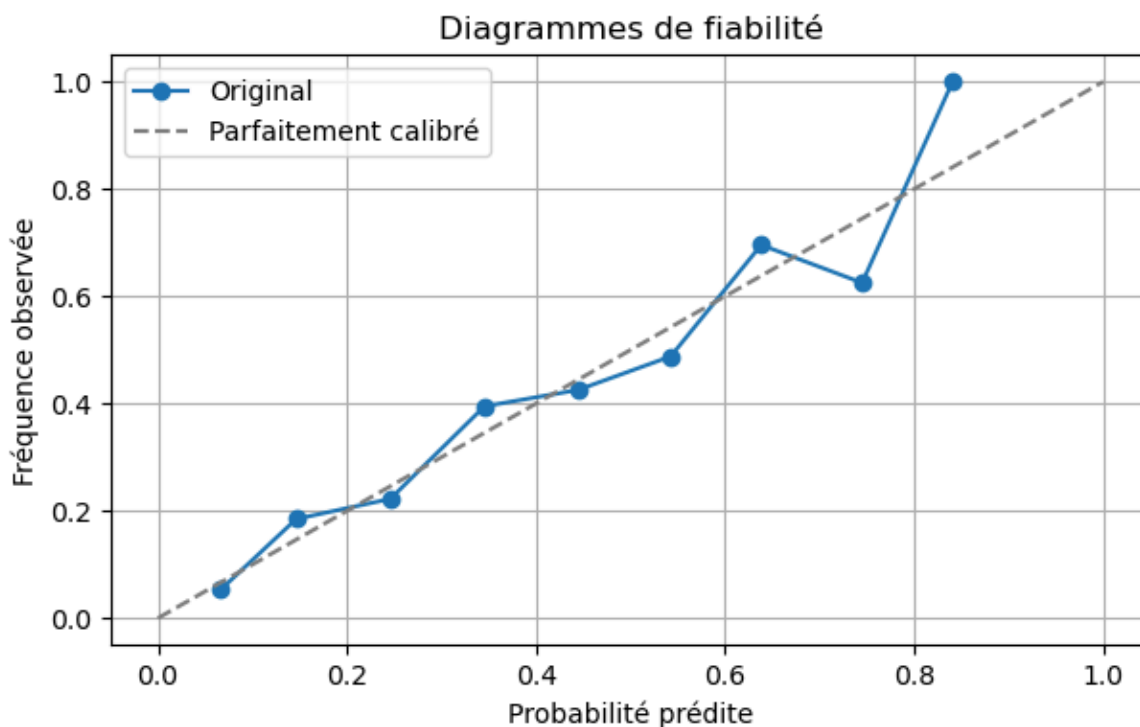
# Evaluation des 3 jeux de probabilités
y_probs_list = [y_prob]# y_prob_platt, y_prob_iso]
names = ["Original", "Platt", "Isotonic"]

# 1. Diagramme de fiabilité
plot_reliability_diagram(y_test, y_probs_list, names)

# 2. Hosmer-Lemeshow, 3. Brier Score, 4. Surconfiance
for probs, label in zip(y_probs_list, names):
    hl_stat, hl_p = hosmer_lemeshow(y_test, probs)
    brier = brier_score_loss(y_test, probs)
    surconf = indice_surconfiance(y_test, probs)

    print(f"--- {label} ---")
    print(f"Hosmer-Lemeshow: Stat={hl_stat:.3f}, p-value={hl_p:.4f}")
    print(f"Brier Score      : {brier:.4f}")
    print(f"Surconfiance      : {surconf:.4f} {'(sur)' if surconf > 0 else '(sous)'}")
    print()

```



```

--- Original ---
Hosmer-Lemeshow: Stat=7.641, p-value=0.4693
Brier Score      : 0.1502
Surconfiance     : -0.0083 (sous)-confiance

```

Recalibration des probabilités : Platt Scaling et Régression Isotonique

Ce bloc applique deux méthodes de **recalibration des probabilités** au modèle de régression logistique initial. L'objectif est d'ajuster les probabilités prédites pour qu'elles reflètent mieux les fréquences observées (amélioration de la calibration).

1. Platt Scaling (`method='sigmoid'`) Cette méthode ajuste les probabilités en les passant dans une **nouvelle régression logistique** (sigmoïde) entraînée sur les

prédictions du modèle initial. Elle est bien adaptée lorsque les données sont peu nombreuses ou la courbe de calibration est lisse.

2. Régression Isotonique (`method='isotonic'`) Cette méthode ajuste les probabilités en utilisant une **fonction monotone non paramétrique** (plus flexible). Elle est utile quand on veut suivre au plus près les fréquences observées, mais peut surajuster si les données sont peu nombreuses.
- Le paramètre `cv=5` signifie que la recalibration est effectuée à l'aide d'une **validation croisée à 5 plis**, pour éviter le surajustement.
 - Les deux modèles (`platt_model` , `iso_model`) peuvent ensuite être utilisés pour prédire des probabilités recalibrées.

```
In [11]: #Recalibrer Les probabilités (Platt Scaling & Isotonic)

from sklearn.calibration import CalibratedClassifierCV

# Platt Scaling
platt_model = CalibratedClassifierCV(base_estimator=model, method='sigmoid', cv=
platt_model.fit(X_train_imputed, y_train)

# Isotonic Regression
iso_model = CalibratedClassifierCV(base_estimator=model, method='isotonic', cv=5
iso_model.fit(X_train_imputed, y_train)
```

```
Out[11]: CalibratedClassifierCV(base_estimator=LogisticRegression(max_iter=1000), cv=5,
method='isotonic')
```

Comparaison visuelle des modèles calibrés

Ce bloc compare la **calibration** de trois modèles :

- le modèle **original** non recalibré,
- le modèle recalibré avec **Platt Scaling**,
- le modèle recalibré avec **régression isotonique**.

Étapes :

- On commence par générer les **nouvelles probabilités prédictives** sur les données de test pour les modèles recalibrés (`y_prob_platt` et `y_prob_iso`).
- Ensuite, on utilise `CalibrationDisplay.from_predictions()` pour afficher sur un même graphique les **trois courbes de calibration**.

Interprétation :

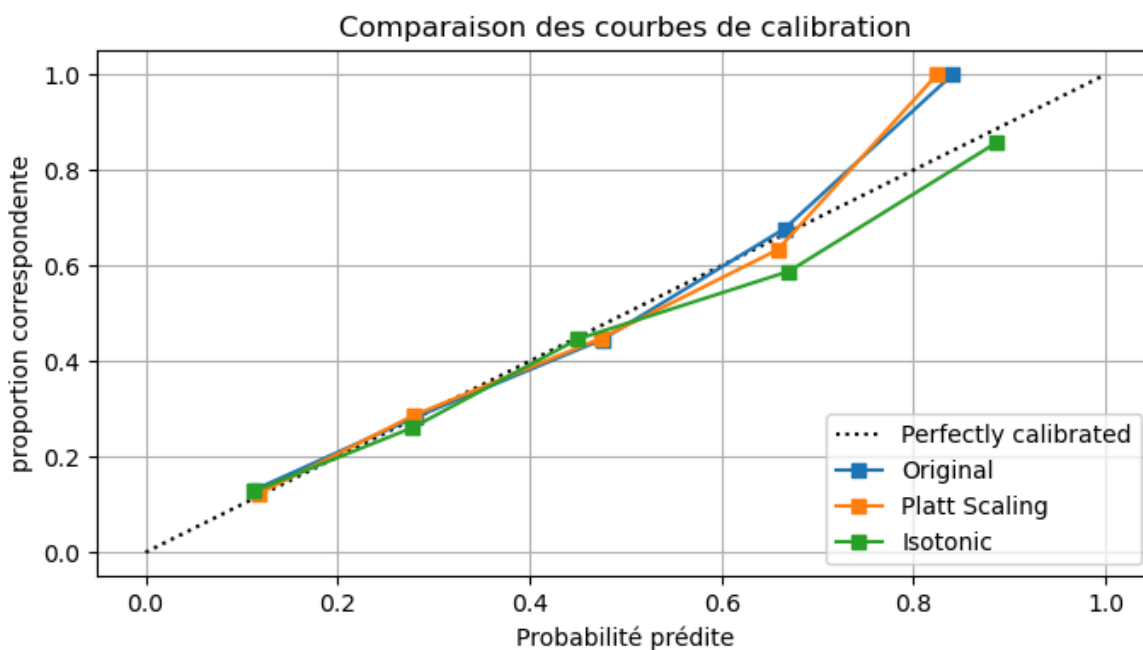
- La diagonale représente une **calibration parfaite**.
- Un modèle est bien calibré si sa courbe est proche de cette diagonale.
- Ce graphique permet de comparer l'effet des recalibrations sur la qualité de prédiction des probabilités.

```
In [12]: #Comparer Les modèles calibrés
# Probabilités des nouveaux modèles
y_prob_platt = platt_model.predict_proba(X_test_imputed)[: , 1]
y_prob_iso = iso_model.predict_proba(X_test_imputed)[: , 1]
```

```
# Courbes de calibration comparatives
plt.figure(figsize=(8, 4))

CalibrationDisplay.from_predictions(y_test, y_prob, name="Original", ax=plt.gca())
CalibrationDisplay.from_predictions(y_test, y_prob_platt, name="Platt Scaling",
                                   ax=plt.gca())
CalibrationDisplay.from_predictions(y_test, y_prob_iso, name="Isotonic", ax=plt.gca())

plt.title("Comparaison des courbes de calibration")
plt.xlabel("Probabilité prédite")
plt.ylabel("proportion correspondante")
plt.grid()
plt.show()
```



Évaluation quantitative détaillée de la calibration

Ce bloc complète l'analyse de calibration avec plusieurs mesures statistiques et graphiques, pour comparer le modèle original et les modèles recalibrés.

1. Diagramme de fiabilité personnalisé

- La fonction `plot_reliability_diagram()` trace, sur un même graphique, les diagrammes de fiabilité des différentes versions du modèle.
- Chaque courbe montre la relation entre les probabilités prédites (`Probabilité prédite`) et la fréquence réelle observée (`Fréquence observée`).
- La diagonale en pointillés représente une calibration parfaite.

2. Test d'Hosmer-Lemeshow

- Ce test statistique (`hosmer_lemeshow()`) évalue la différence entre le nombre observé de défauts et le nombre attendu dans chaque groupe.
- La **statistique de test** et la **p-value** permettent de savoir si le modèle est bien calibré.
- Une p-value élevée (> 0.05) suggère que le modèle n'est pas significativement différent d'un modèle parfaitement calibré.

3. Brier Score

- Le Brier Score est une mesure de la qualité des probabilités prédites. Plus il est bas, meilleure est la qualité.

4. Indice de sur/sous-confiance

- L'indice mesure si le modèle est globalement **trop confiant** (surconfiance) ou **pas assez** (sous-confiance).
- Une valeur positive indique une surévaluation des risques.

Application pratique

- Ces trois mesures sont calculées et affichées pour chaque modèle : `Original`, `Platt`, et `Isotonic`.
- Cette évaluation combinée permet de choisir la meilleure méthode de recalibration.

```
In [13]: from sklearn.calibration import calibration_curve
from sklearn.metrics import brier_score_loss
from scipy.stats import chi2

# Fonction pour Le diagramme de fiabilité
def plot_reliability_diagram(y_true, prob_sets, names):
    plt.figure(figsize=(7, 5))
    for probs, label in zip(prob_sets, names):
        prob_true, prob_pred = calibration_curve(y_true, probs, n_bins=10, strat
        plt.plot(prob_pred, prob_true, marker='o', label=label)
    plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Parfaitement c
    plt.xlabel('Probabilité prédite')
    plt.ylabel('Fréquence observée')
    plt.title('Diagrammes de fiabilité comparés')
    plt.legend()
    plt.grid()
    plt.show()

# Fonction pour Le test d'Hosmer-Lemeshow
def hosmer_lemeshow(y_true, y_prob, g=10):
    data = pd.DataFrame({'y': y_true, 'prob': y_prob})
    data['bucket'] = pd.qcut(data['prob'], q=g, duplicates='drop')
    obs = data.groupby('bucket')['y'].sum()
    exp = data.groupby('bucket')['prob'].sum()
    n = data.groupby('bucket')['y'].count()
    hl_stat = ((obs - exp)**2 / (exp * (1 - exp / n))).sum()
    df = g - 2
    p_value = 1 - chi2.cdf(hl_stat, df)
    return hl_stat, p_value

# Fonction pour L'indice de sur/sous-confiance
def indice_surconfiance(y_true, y_prob):
    prob_true, prob_pred = calibration_curve(y_true, y_prob, n_bins=10, strategy
    return np.mean(prob_pred - prob_true)

# Evaluation des 3 jeux de probabilités
y_probs_list = [y_prob, y_prob_platt, y_prob_iso]
names = ["Original", "Platt", "Isotonic"]

# 1. Diagramme de fiabilité
```

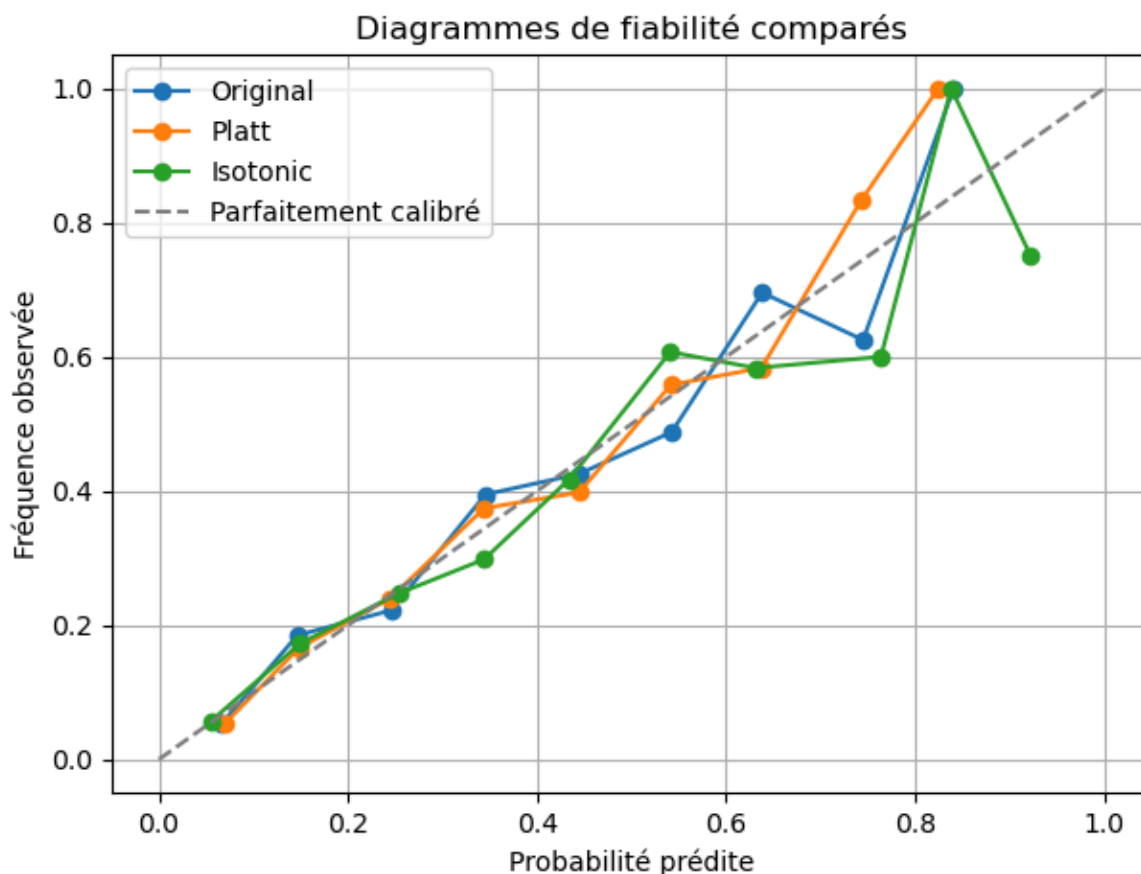
```

plot_reliability_diagram(y_test, y_probs_list, names)

# 2. Hosmer-Lemeshow, 3. Brier Score, 4. Surconfiance
for probs, label in zip(y_probs_list, names):
    hl_stat, hl_p = hosmer_lemeshow(y_test, probs)
    brier = brier_score_loss(y_test, probs)
    surconf = indice_surconfiance(y_test, probs)

    print(f"--- {label} ---")
    print(f"Hosmer-Lemeshow: Stat={hl_stat:.3f}, p-value={hl_p:.4f}")
    print(f"Brier Score      : {brier:.4f}")
    print(f"Surconfiance      : {surconf:.4f} {'(sur)' if surconf > 0 else '(sous)'}")
    print()

```



```

--- Original ---
Hosmer-Lemeshow: Stat=7.641, p-value=0.4693
Brier Score      : 0.1502
Surconfiance     : -0.0083 (sous)-confiance

```

```

--- Platt ---
Hosmer-Lemeshow: Stat=8.182, p-value=0.4158
Brier Score      : 0.1498
Surconfiance     : -0.0232 (sous)-confiance

```

```

--- Isotonic ---
Hosmer-Lemeshow: Stat=8.135, p-value=0.4204
Brier Score      : 0.1496
Surconfiance     : 0.0199 (sur)-confiance

```

Optimisation des seuils de décision via précision, rappel, F1-score et coût métier

Ce bloc a pour objectif de déterminer le seuil optimal pour convertir les probabilités de défaut en décisions binaires (accepté/refusé) en équilibrant différents critères :

1. Calcul de la courbe Précision-Rappel :

- On calcule la précision et le rappel pour différents seuils sur les probabilités recalibrées (`y_prob_platt`).
- Le F1-score, qui est la moyenne harmonique entre précision et rappel, est calculé pour chaque seuil.

2. Choix du seuil optimal basé sur le F1-score :

- Le seuil donnant le F1-score maximal est identifié (`optimal_threshold_f1`).
- Ce seuil équilibre bien précision et rappel, ce qui est souvent un bon compromis en classification binaire.

3. Visualisation des courbes :

- **Précision vs Rappel** : permet de visualiser le compromis entre ces deux métriques.
- **Précision, Rappel et F1 vs seuil** : montre comment évoluent ces métriques selon le seuil de décision.
- **Coût total vs seuil** : évalue le coût métier associé à chaque seuil.

4. Calcul du seuil optimal selon la minimisation du coût métier :

- La fonction `calculate_cost` calcule un coût total prenant en compte le coût d'un faux négatif (défaut non détecté) et d'un faux positif (refus injustifié).
- Le seuil minimisant ce coût est déterminé (`optimal_threshold_cost`).

5. Affichage des performances finales :

- On affiche les matrices de confusion et les scores F1 aux seuils optimaux choisis selon F1 et selon coût.

Ce processus aide à choisir un seuil adapté aux objectifs business, qu'ils soient purement techniques (F1) ou économiques (coût métier).

```
In [14]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import precision_recall_curve, f1_score, confusion_matrix

# 1. Courbe Précision-Rappel et F1-Score
precision, recall, thresholds = precision_recall_curve(y_test, y_prob_platt)
f1_scores = 2 * (precision * recall) / (precision + recall + 1e-9)

# Seuil optimal pour F1-Score
optimal_idx = np.argmax(f1_scores[:-1])
optimal_threshold_f1 = thresholds[optimal_idx]

# 2. Trace des courbes
plt.figure(figsize=(15, 6))

# --- Tracé de La courbe de Précision vs Rappel ---
plt.subplot(1, 3, 1)
plt.plot(recall, precision, label='Courbe Précision-Rappel')
plt.scatter(recall[optimal_idx], precision[optimal_idx], color='red', s=100, lab
```

```

plt.xlabel('Rappel')
plt.ylabel('Précision')
plt.legend()
plt.title('Courbe de Trade-off Précision/Rappel')
plt.grid(True)

# --- Tracé de La courbe Précision, Rappel et F1 vs seuil (comme dans votre code)
plt.subplot(1, 3, 2)
plt.plot(thresholds, precision[:-1], label='Précision')
plt.plot(thresholds, recall[:-1], label='Rappel')
plt.plot(thresholds, f1_scores[:-1], label='F1-Score', linestyle='--')
plt.axvline(optimal_threshold_f1, color='r', linestyle=':', label=f'Seuil F1 opt')
plt.xlabel('Seuil de Décision')
plt.ylabel('Score')
plt.legend()
plt.title('Précision, Rappel, F1 vs Seuil')
plt.grid(True)

# --- Tracé de La courbe de Coût (comme dans votre code original) ---
def calculate_cost(y_true, y_prob_platt, threshold, cost_fn=5, cost_fp=1):
    y_pred = (y_prob_platt >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    return fn * cost_fn + fp * cost_fp

costs = [calculate_cost(y_test, y_prob_platt, t) for t in thresholds]
optimal_threshold_cost = thresholds[np.argmin(costs)]

plt.subplot(1, 3, 3)
plt.plot(thresholds, costs, label='Coût Total')
plt.axvline(optimal_threshold_cost, color='r', linestyle=':', label=f'Seuil Coût')
plt.xlabel('Seuil de Décision')
plt.ylabel('Coût Total')
plt.legend()
plt.title('Minimisation du Coût Métier')
plt.grid(True)

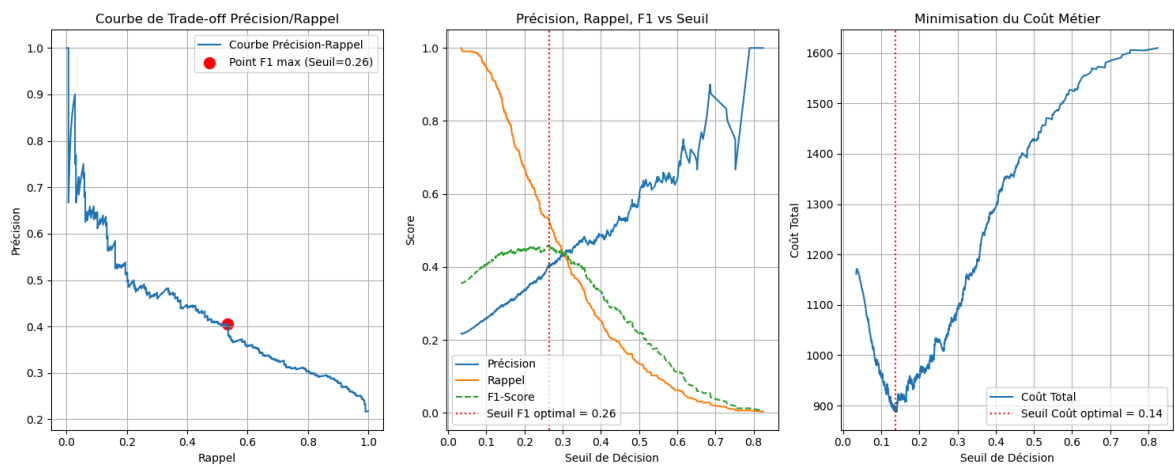
plt.tight_layout()
plt.show()

# Métriques finales (restent les mêmes)
y_pred_f1 = (y_prob_platt >= optimal_threshold_f1).astype(int)
y_pred_cost = (y_prob_platt >= optimal_threshold_cost).astype(int)

print("\n--- Performances avec le seuil F1 optimal ---")
print(f"Seuil = {optimal_threshold_f1:.2f}")
print(f"F1-Score = {f1_score(y_test, y_pred_f1):.3f}")
print(confusion_matrix(y_test, y_pred_f1))

print("\n--- Performances avec le seuil de Coût optimal ---")
print(f"Seuil = {optimal_threshold_cost:.2f}")
print(f"Coût Total = {min(costs):.1f}")
print(confusion_matrix(y_test, y_pred_cost))

```



--- Performances avec le seuil F1 optimal ---

Seuil = 0.26

F1-Score = 0.460

[[924 253]

[151 172]]

--- Performances avec le seuil de Coût optimal ---

Seuil = 0.14

Coût Total = 888.0

[[494 683]

[41 282]]

In []:

```
In [15]: # 1. Courbe Précision-Rappel et F1-Score
precision, recall, thresholds = precision_recall_curve(y_test, y_prob_platt)
f1_scores = 2 * (precision * recall) / (precision + recall + 1e-9) # Éviter les

# Seuil optimal pour F1-Score
optimal_idx = np.argmax(f1_scores[:-1]) # Ignorer le dernier point (non pertinent)
optimal_threshold_f1 = thresholds[optimal_idx]

# 2. Minimisation du Coût Métier (exemple : Coût_FN = 5, Coût_FP = 1)
def calculate_cost(y_true, y_prob_platt, threshold, cost_fn=5, cost_fp=1):
    y_pred = (y_prob_platt >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    return fn * cost_fn + fp * cost_fp

costs = [calculate_cost(y_test, y_prob_platt, t) for t in thresholds]
optimal_threshold_cost = thresholds[np.argmin(costs)]

# 3. Affichage des résultats
plt.figure(figsize=(12, 6))

# Courbe Précision-Rappel
plt.subplot(1, 2, 1)
plt.plot(thresholds, precision[:-1], label='Précision')
plt.plot(thresholds, recall[:-1], label='Rappel')
plt.plot(thresholds, f1_scores[:-1], label='F1-Score', linestyle='--')
plt.axvline(optimal_threshold_f1, color='r', linestyle=':', label=f'Seuil F1 opt')
plt.xlabel('Seuil de Décision')
plt.ylabel('Score')
plt.legend()
plt.title('Trade-off Précision-Rappel')

# Courbe de Coût
```

```

plt.subplot(1, 2, 2)
plt.plot(thresholds, costs, label='Coût Total')
plt.axvline(optimal_threshold_cost, color='r', linestyle=':', label=f'Seuil Coût')
plt.xlabel('Seuil de Décision')
plt.ylabel('Coût Total')
plt.legend()
plt.title('Minimisation du Coût Métier')

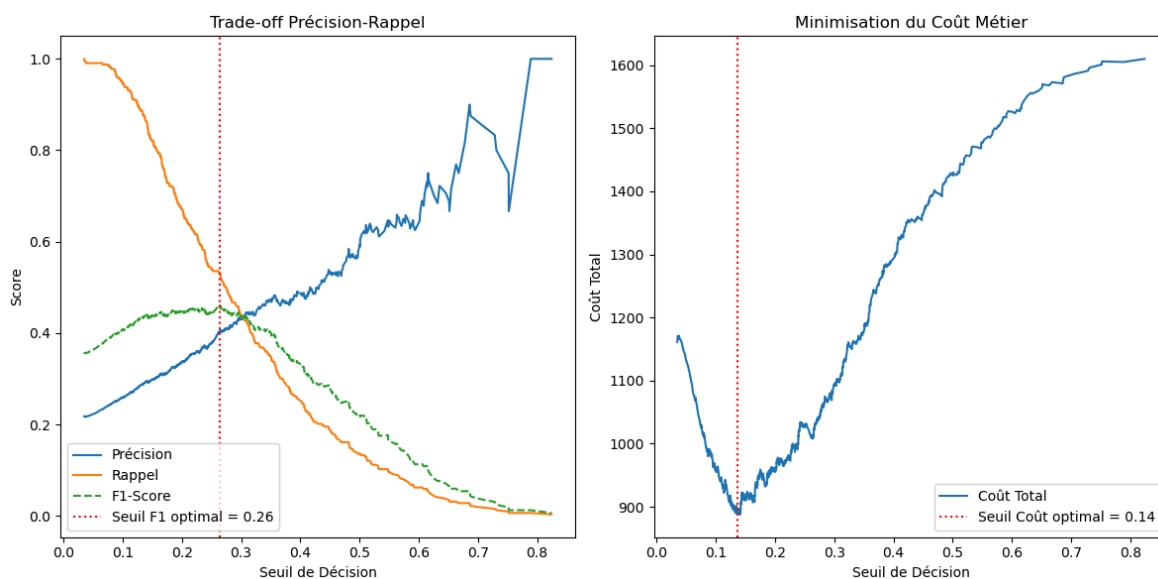
plt.tight_layout()
plt.show()

# Métriques finales
y_pred_f1 = (y_prob_platt >= optimal_threshold_f1).astype(int)
y_pred_cost = (y_prob_platt >= optimal_threshold_cost).astype(int)

print("\n--- Performances avec le seuil F1 optimal ---")
print(f"Seuil = {optimal_threshold_f1:.2f}")
print(f"F1-Score = {f1_score(y_test, y_pred_f1):.3f}")
print(confusion_matrix(y_test, y_pred_f1))

print("\n--- Performances avec le seuil de Coût optimal ---")
print(f"Seuil = {optimal_threshold_cost:.2f}")
print(f"Coût Total = {min(costs):.1f}")
print(confusion_matrix(y_test, y_pred_cost))

```



--- Performances avec le seuil F1 optimal ---

Seuil = 0.26

F1-Score = 0.460

[[924 253]

[151 172]]

--- Performances avec le seuil de Coût optimal ---

Seuil = 0.14

Coût Total = 888.0

[[494 683]

[41 282]]

Construction et visualisation du front de Pareto : Précision vs Rappel

Ce bloc explore l'équilibre entre **précision** et **rappel** pour un large éventail de seuils de décision sur les probabilités recalibrées (`y_prob_platt`).

- On génère une liste de seuils uniformément répartis entre 0 et 1.
- Pour chaque seuil, on calcule les prédictions binaires (`y_pred`) en comparant les probabilités au seuil.
- Si aucune prédiction positive n'est faite (pour éviter division par zéro), on saute ce seuil.
- Pour chaque seuil retenu, on calcule les métriques clés :
 - **Précision** : proportion des prédictions positives correctes.
 - **Rappel** : proportion des positifs réels détectés.
 - **F1-score** : moyenne harmonique précision-rappel.
 - **Taux d'acceptation** : proportion d'individus acceptés (prédits négatifs ici, selon le contexte).
- Ces résultats sont stockés dans un tableau (`df1`) pour un usage et une analyse ultérieurs.
- Enfin, on trace le **front de Pareto** en représentant la précision en fonction du rappel pour chaque seuil, visualisant ainsi le compromis possible entre ces deux métriques.

Ce graphique est précieux pour choisir un seuil adapté selon les priorités métier entre rappel (détection des mauvais payeurs) et précision (limiter les faux positifs).

```
In [16]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import precision_score, recall_score, f1_score

thresholds = np.linspace(0, 1, 200)
results = []

for threshold in thresholds:
    y_pred = (y_prob_platt >= threshold).astype(int)

    if np.sum(y_pred) == 0: # Évite les divisions par 0
        continue

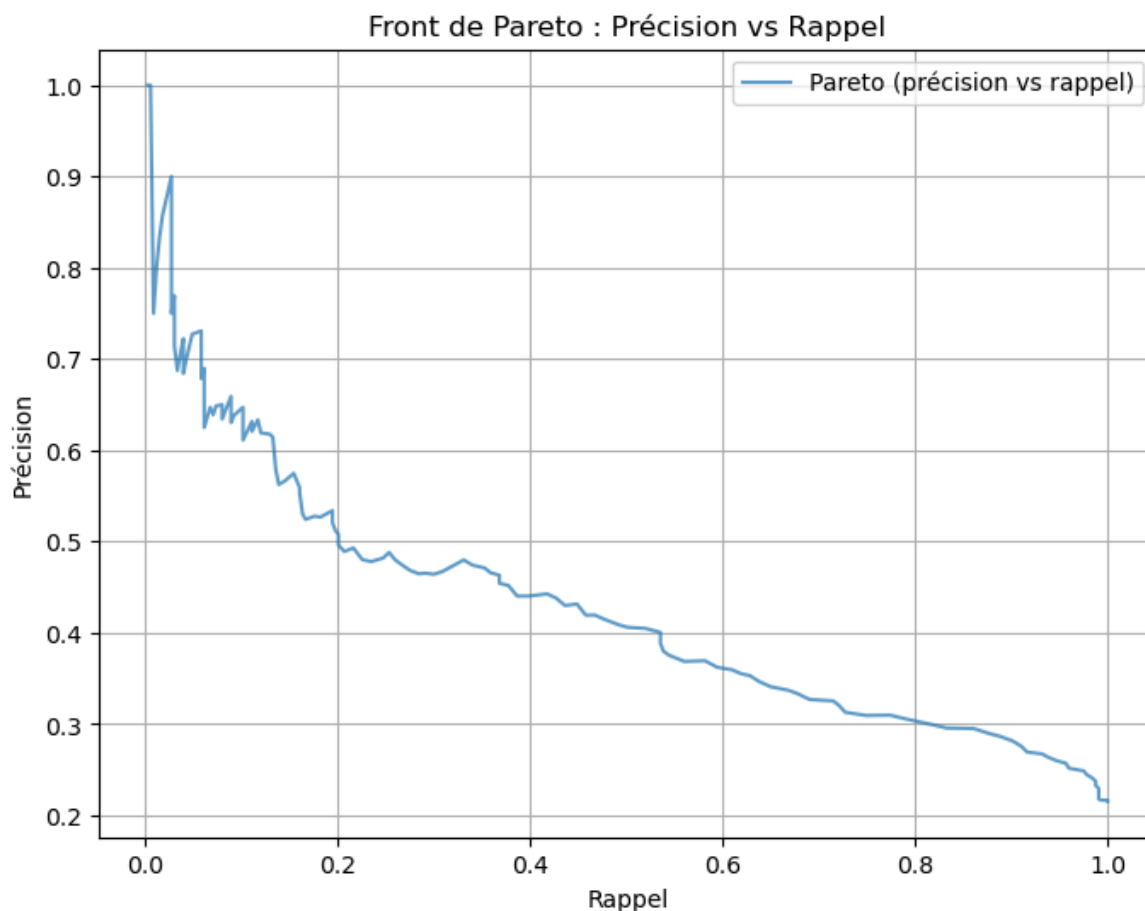
    precision = precision_score(y_test, y_pred, zero_division=0)
    recall = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    acceptance_rate = np.mean(y_pred)

    results.append({
        'seuil': threshold,
        'precision': precision,
        'recall': recall,
        'f1': f1,
        'acceptance_rate': acceptance_rate
    })

# Convertir en tableau pour usage ultérieur
import pandas as pd
df1 = pd.DataFrame(results)

# Tracer le front de Pareto
plt.figure(figsize=(8, 6))
plt.plot(df1['recall'], df1['precision'], label='Pareto (précision vs rappel)',
plt.xlabel('Rappel')
plt.ylabel('Précision')
```

```
plt.title('Front de Pareto : Précision vs Rappel')
plt.grid(True)
plt.legend()
plt.show()
```



Identification et visualisation du front de Pareto

Ce bloc permet de détecter et visualiser les **points optimaux** dans le compromis entre précision et rappel, appelés le **front de Pareto**.

- La fonction `is_pareto_efficient()` identifie les points dits **non dominés**, c'est-à-dire ceux pour lesquels il n'existe aucun autre point qui améliore à la fois la précision et le rappel.
- Elle retourne un tableau booléen indiquant quels points appartiennent au front.
- Ensuite, on ajoute une colonne `is_pareto` au DataFrame pour marquer ces points.
- Enfin, on trace :
 - Tous les points pour tous les seuils en bleu clair,
 - Les points du front de Pareto en rouge avec un symbole différent (`x`).

Ce graphique aide à sélectionner des seuils qui offrent le meilleur compromis entre précision et rappel sans être dominés par d'autres configurations.

```
In [17]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import precision_score, recall_score, f1_score

# Supposons que y_test et y_prob_platt sont déjà définis
```

```

# Exemple de données (remplacez par vos propres données)

# Définition de la fonction is_pareto_efficient
def is_pareto_efficient(costs, return_mask=True):
    is_efficient = np.ones(costs.shape[0], dtype=bool)
    for i, c in enumerate(costs):
        if is_efficient[i]:
            is_efficient[is_efficient] = np.any(costs[is_efficient] >= c, axis=1)
            is_efficient[i] = True
    return is_efficient

# 1. Calcul des métriques pour chaque seuil
thresholds = np.linspace(0, 1, 200)
results = []

for threshold in thresholds:
    y_pred = (y_prob_platt >= threshold).astype(int)

    if np.sum(y_pred) == 0: # Évite les divisions par 0
        continue

    precision = precision_score(y_test, y_pred, zero_division=0)
    recall = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    acceptance_rate = np.mean(y_pred)

    results.append({
        'seuil': threshold,
        'precision': precision,
        'recall': recall,
        'f1': f1,
        'acceptance_rate': acceptance_rate
    })

df1 = pd.DataFrame(results)

# 2. Identification du front de Pareto
# On combine les colonnes precision et recall pour les passer à la fonction
costs = df1[['precision', 'recall']].to_numpy()
is_pareto = is_pareto_efficient(costs)

# Ajout de la colonne au DataFrame
df1['is_pareto'] = is_pareto

# 3. Visualisation du front de Pareto
plt.figure(figsize=(8, 5))

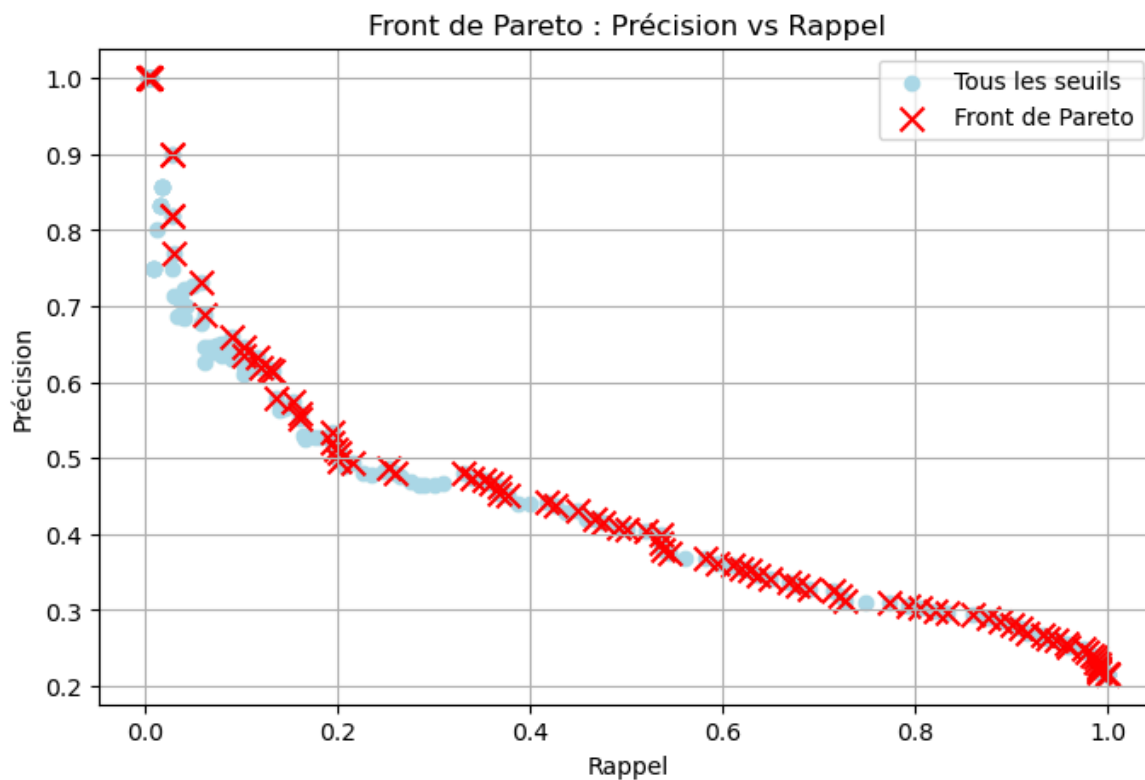
# Tous les points
plt.scatter(df1['recall'], df1['precision'], color='lightblue', label='Tous les')

# Points du front de Pareto
pareto_front = df1[df1['is_pareto']]
plt.scatter(pareto_front['recall'], pareto_front['precision'],
            color='red', marker='x', s=100, label='Front de Pareto')

plt.xlabel('Rappel')
plt.ylabel('Précision')
plt.title('Front de Pareto : Précision vs Rappel')
plt.grid(True)

```

```
plt.legend()
plt.show()
```



```
In [18]: # 1. Calcul des métriques pour chaque seuil
# ... (votre code ici) ...
df1 = pd.DataFrame(results)

# 2. Identification du front de Pareto
# On combine les colonnes precision et recall pour les passer à la fonction
costs = df1[['precision', 'recall']].to_numpy()
is_pareto = is_pareto_efficient(costs)
df1['is_pareto'] = is_pareto

# 3. Filtrer le DataFrame pour ne garder que les points du front de Pareto
pareto_front = df1[df1['is_pareto']]

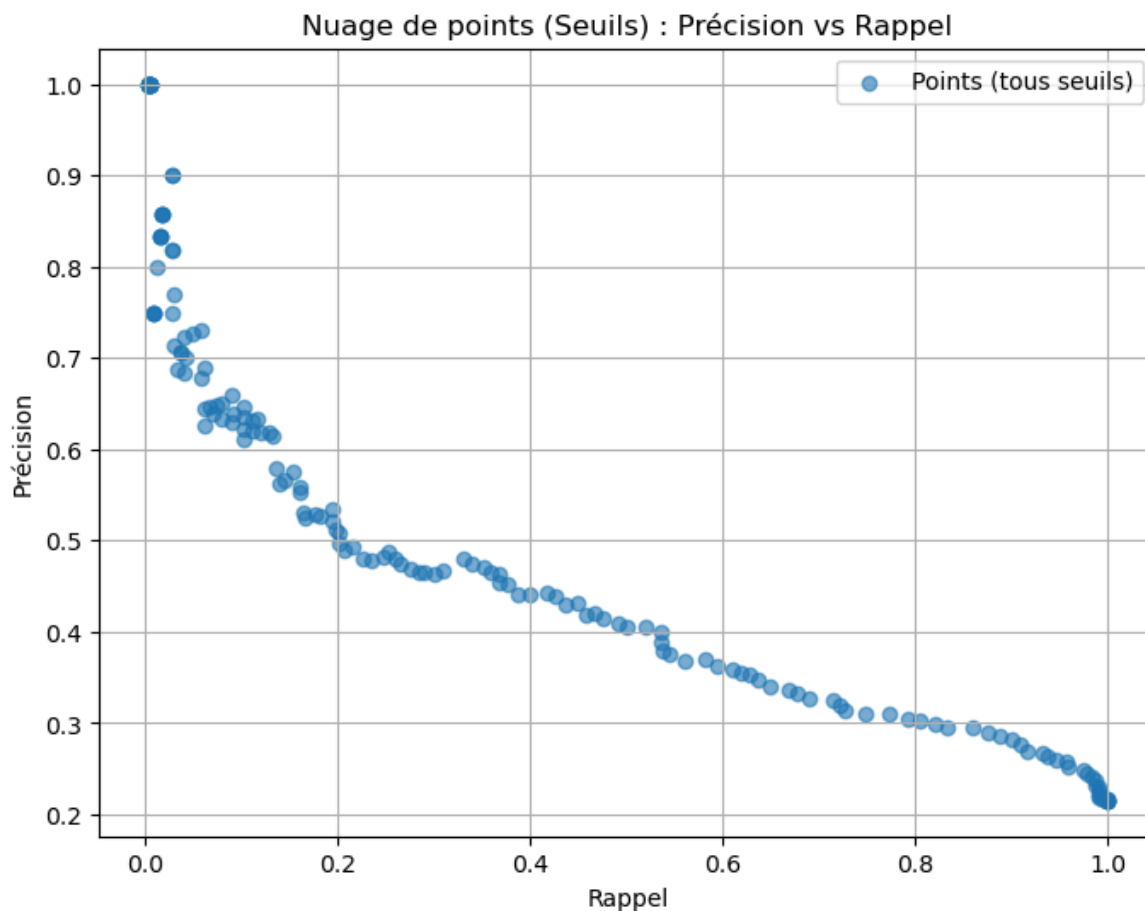
# 4. Afficher le tableau avec les seuils, la précision et le rappel
print("Tableau des seuils du front de Pareto:")
print(pareto_front[['seuil', 'precision', 'recall']].to_string(index=False))
```

Tableau des seuils du front de Pareto:

seuil	precision	recall
0.000000	0.215333	1.000000
0.005025	0.215333	1.000000
0.010050	0.215477	1.000000
0.015075	0.215477	1.000000
0.020101	0.215621	1.000000
0.025126	0.216054	1.000000
0.030151	0.216633	1.000000
0.040201	0.217687	0.990712
0.045226	0.220234	0.990712
0.050251	0.222841	0.990712
0.055276	0.225670	0.990712
0.060302	0.229555	0.990712
0.065327	0.232507	0.987616
0.070352	0.237175	0.987616
0.075377	0.240545	0.984520
0.080402	0.244582	0.978328
0.085427	0.248619	0.975232
0.090452	0.251623	0.959752
0.095477	0.257072	0.956656
0.100503	0.259762	0.947368
0.105528	0.263708	0.938080
0.110553	0.267318	0.931889
0.115578	0.269336	0.916409
0.120603	0.275797	0.910217
0.125628	0.281704	0.900929
0.130653	0.286427	0.888545
0.135678	0.289959	0.876161
0.140704	0.295117	0.860681
0.145729	0.295604	0.832817
0.150754	0.298760	0.820433
0.155779	0.302326	0.804954
0.160804	0.305125	0.792570
0.165829	0.309789	0.773994
0.175879	0.312916	0.727554
0.180905	0.320055	0.721362
0.185930	0.325352	0.715170
0.190955	0.326979	0.690402
0.195980	0.333333	0.678019
0.201005	0.336973	0.668731
0.206030	0.340909	0.650155
0.211055	0.346801	0.637771
0.216080	0.353043	0.628483
0.221106	0.355240	0.619195
0.226131	0.359489	0.609907
0.231156	0.362264	0.594427
0.236181	0.369352	0.582043
0.246231	0.375267	0.544892
0.251256	0.379913	0.538700
0.256281	0.388764	0.535604
0.261307	0.400463	0.535604
0.266332	0.404819	0.520124
0.271357	0.406015	0.501548
0.276382	0.408740	0.492260
0.281407	0.415094	0.476780
0.286432	0.419444	0.467492
0.296482	0.431548	0.448916
0.306533	0.438095	0.427245
0.311558	0.442623	0.417957

0.326633	0.451852	0.377709
0.331658	0.454198	0.368421
0.336683	0.463035	0.368421
0.341709	0.465863	0.359133
0.346734	0.471074	0.352941
0.351759	0.474138	0.340557
0.356784	0.479821	0.331269
0.391960	0.480000	0.260062
0.396985	0.488095	0.253870
0.417085	0.492958	0.216718
0.427136	0.496183	0.201238
0.432161	0.507812	0.201238
0.437186	0.512000	0.198142
0.442211	0.520661	0.195046
0.447236	0.533898	0.195046
0.472362	0.553191	0.160991
0.477387	0.559140	0.160991
0.482412	0.574713	0.154799
0.497487	0.578947	0.136223
0.502513	0.614286	0.133127
0.507538	0.617647	0.130031
0.512563	0.619048	0.120743
0.517588	0.633333	0.117647
0.542714	0.634615	0.102167
0.547739	0.647059	0.102167
0.562814	0.659091	0.089783
0.603015	0.689655	0.061920
0.613065	0.730769	0.058824
0.658291	0.769231	0.030960
0.668342	0.818182	0.027864
0.673367	0.818182	0.027864
0.678392	0.900000	0.027864
0.683417	0.900000	0.027864
0.753769	1.000000	0.006192
0.758794	1.000000	0.006192
0.763819	1.000000	0.006192
0.768844	1.000000	0.006192
0.773869	1.000000	0.006192
0.778894	1.000000	0.006192
0.783920	1.000000	0.006192
0.788945	1.000000	0.006192
0.793970	1.000000	0.003096
0.798995	1.000000	0.003096
0.804020	1.000000	0.003096
0.809045	1.000000	0.003096
0.814070	1.000000	0.003096
0.819095	1.000000	0.003096
0.824121	1.000000	0.003096

```
In [19]: plt.figure(figsize=(8, 6))
plt.scatter(df1['recall'], df1['precision'], alpha=0.6, label='Points (tous seuils)')
plt.xlabel('Rappel')
plt.ylabel('Précision')
plt.title('Nuage de points (Seuils) : Précision vs Rappel')
plt.grid(True)
plt.legend()
plt.show()
```



Optimisation sous contrainte : taux d'acceptation minimal

Ce bloc applique une contrainte métier importante sur le **taux d'acceptation** (proportion de clients acceptés).

- On filtre les seuils pour ne conserver que ceux où le **taux d'acceptation est au moins égal à 40%** (`acceptance_rate >= 0.4`).
- Parmi ces seuils compatibles avec la contrainte, on sélectionne celui qui maximise le **F1-score**, garantissant ainsi le meilleur compromis précision/rappel possible tout en respectant la contrainte business.
- Le seuil optimal ainsi obtenu est affiché avec ses métriques associées.

Cette démarche est essentielle pour assurer que la stratégie de scoring ne soit pas trop restrictive et permette un volume d'acceptation suffisant.

```
In [20]: # Filtrer les seuils qui respectent la contrainte business
df_contrainte = df1[df1['acceptance_rate'] >= 0.4]

# Identifier le meilleur F1-score parmi ceux qui respectent la contrainte
best_point = df_contrainte.loc[df_contrainte['f1'].idxmax()]

print(" Meilleur seuil sous contrainte (≥ 40% acceptation) :")
print(best_point)
```

```

Meilleur seuil sous contrainte ( $\geq 40\%$  acceptation) :
seuil          0.201005
precision      0.336973
recall         0.668731
f1             0.448133
acceptance_rate 0.427333
is_pareto      True
Name: 40, dtype: object

```

Analyse de stabilité des performances par bootstrap

Ce bloc évalue la **robustesse** du seuil optimal choisi face aux variations aléatoires des données :

- On utilise la méthode de **bootstrap** pour simuler 30 sous-échantillons aléatoires (avec remise) à partir du jeu de test.
- Pour chaque échantillon bootstrap, on calcule les prédictions binaires avec le seuil optimal (`seuil`) et on mesure le **F1-score**.
- Cela permet d'observer la variabilité des performances du modèle sous ce seuil face aux fluctuations possibles des données réelles.
- Enfin, on affiche la **moyenne** des F1-scores (performance moyenne) et leur **écart-type** (mesure de stabilité).

Cette analyse est importante pour valider que la sélection du seuil est fiable et que le modèle ne risque pas d'avoir des performances très variables.

```

In [21]: from sklearn.utils import resample

seuil = best_point['seuil']
f1_scores = []

# Bootstrap : simule des fluctuations de données
for i in range(30):
    y_test_boot, y_prob_boot = resample(y_test, y_prob_platt, random_state=i)
    y_pred_boot = (y_prob_boot >= seuil).astype(int)
    f1_boot = f1_score(y_test_boot, y_pred_boot, zero_division=0)
    f1_scores.append(f1_boot)

# Analyser la stabilité
print(f" Moyenne F1-score : {np.mean(f1_scores):.3f}")
print(f" Écart-type (stabilité) : {np.std(f1_scores):.3f}")

```

Moyenne F1-score : 0.453

Écart-type (stabilité) : 0.025

Analyse de sensibilité du F1-score autour du seuil optimal

Ce bloc examine comment la performance (F1-score) du modèle varie lorsque le seuil de décision est légèrement modifié autour de la valeur optimale choisie.

- On définit un intervalle autour du seuil optimal (`seuil  $\pm$  0.05`), avec des pas de 0.005.
- Pour chaque seuil dans cet intervalle, on calcule les prédictions binaires et le F1-score correspondant.

- Les résultats sont stockés dans un DataFrame pour faciliter la visualisation.
- On trace la courbe F1-score en fonction du seuil, avec une ligne verticale indiquant le seuil optimal.

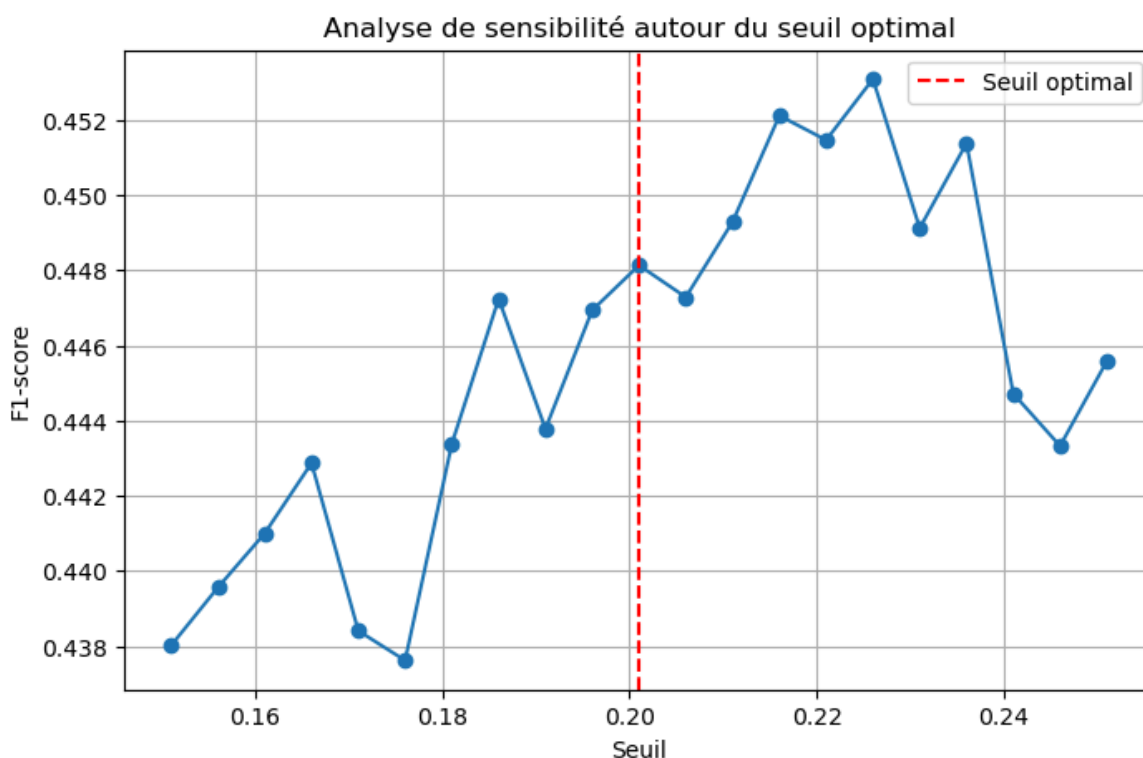
Cette analyse permet de vérifier la **sensibilité** du modèle aux petites variations de seuil, important pour s'assurer que le seuil choisi est stable et robuste.

```
In [22]: seuil_range = np.arange(seuil - 0.05, seuil + 0.05, 0.005)
sensibilite = []

for s in seuil_range:
    y_pred_s = (y_prob_platt >= s).astype(int)
    f1_s = f1_score(y_test, y_pred_s, zero_division=0)
    sensibilite.append((s, f1_s))

# Tracer
sensibilite = pd.DataFrame(sensibilite, columns=['seuil', 'f1'])

plt.figure(figsize=(8, 5))
plt.plot(sensibilite['seuil'], sensibilite['f1'], marker='o')
plt.axvline(best_point['seuil'], color='red', linestyle='--', label='Seuil optimal')
plt.title("Analyse de sensibilité autour du seuil optimal")
plt.xlabel("Seuil")
plt.ylabel("F1-score")
plt.grid(True)
plt.legend()
plt.show()
```



Évaluation finale du modèle avec seuil fixé

Ce bloc calcule plusieurs métriques clés pour évaluer les performances du modèle recalibré :

1. AUC (Area Under the Curve) ROC

- Mesure la capacité globale du modèle à discriminer entre bons et mauvais payeurs, en utilisant les probabilités continues (`y_prob_platt`).
- L'AUC varie entre 0.5 (modèle aléatoire) et 1 (modèle parfait).

2. Conversion des probabilités en prédictions binaires

- On fixe un seuil de décision à 0.20 (exemple ici) pour transformer les probabilités en classes :
 - Probabilité $\geq$ seuil $\rightarrow$ mauvais payeur (classe 1)
 - Probabilité $<$ seuil $\rightarrow$ bon payeur (classe 0)

3. Calcul de la matrice de confusion et du rapport de classification

- La matrice de confusion détaille le nombre de vrais positifs, faux positifs, vrais négatifs, et faux négatifs.
- Le rapport de classification fournit des métriques détaillées (précision, rappel, F1-score) pour chaque classe.

Ce bilan final est utile pour comprendre la performance pratique du modèle selon un seuil donné.

```
In [23]: from sklearn.metrics import roc_auc_score, confusion_matrix, classification_report
import numpy as np

# --- 1. Calcul des métriques qui utilisent des probabilités ---
# roc_auc_score accepte les probabilités
print("AUC :", roc_auc_score(y_test, y_prob_platt))

# --- 2. Convertir les probabilités en prédictions binaires ---
# Appliquer un seuil pour convertir les probabilités continues
seuil = 0.20
y_pred_binary = (y_prob_platt >= seuil).astype(int)

# --- 3. Calcul des métriques qui utilisent des classes binaires ---
# confusion_matrix et classification_report ont besoin de classes binaires
print("Matrice de confusion :\n", confusion_matrix(y_test, y_pred_binary))
print("Rapport de classification :\n", classification_report(y_test, y_pred_bina
```

AUC : 0.7238768869798065

Matrice de confusion :

```
[[752 425]
```

```
[106 217]]
```

Rapport de classification :

	precision	recall	f1-score	support
0	0.88	0.64	0.74	1177
1	0.34	0.67	0.45	323
accuracy			0.65	1500
macro avg	0.61	0.66	0.59	1500
weighted avg	0.76	0.65	0.68	1500

Construction d'un DataFrame de test enrichi avec les probabilités et variables clés

Ce bloc crée un DataFrame combinant les données d'entrée du jeu de test avec :

- Les **probabilités prédites recalibrées** (`y_prob_platt`), pour chaque individu.
- Quelques variables explicatives importantes (`Age` , `Revenus` , `Score_Bureau` , `Anciennete_Emploi`) qui seront utiles pour appliquer des règles décisionnelles segmentées ou analyses ultérieures.

Ce DataFrame `df_test` facilite les analyses personnalisées et la mise en place de stratégies de décision spécifiques par segment.

```
In [24]: # Créer un DataFrame test avec toutes les infos + proba prédite
df_test = X_test_imputed.copy()
df_test['y_prob_platt'] = y_prob_platt
#df_test['Age'] = X_test_imputed['Age'] # utile pour les règles
#df_test['Revenus'] = X_test_imputed['Revenus']
#df_test['Score_Bureau'] = X_test_imputed['Score_Bureau']
#df_test['Anciennete_Emploi'] = X_test_imputed['Anciennete_Emploi']
```

Fonction de décision finale intégrant stratégies avancées

Cette fonction `decision_finale` applique une série de règles combinées pour décider du statut final d'un client selon :

1. Seuils adaptatifs par segment de revenus

- Les seuils de décision varient selon la tranche de revenu pour mieux refléter le risque associé à chaque segment.
- Plus le revenu est faible, plus le seuil est élevé (plus strict).

2. Zone d'incertitude (rejet temporaire)

- Autour du seuil, une zone tampon est définie (± 0.05) où la décision est différée (`REJET TEMPORAIRE`) pour éviter les décisions hâtives sur des cas incertains.

3. Règles hiérarchiques supplémentaires

- Un score de crédit bureau faible (<400) conduit à un refus automatique.
- Une ancienneté d'emploi élevée (>10 ans) combinée à un revenu suffisant (>2500) peut faire accepter même si la probabilité est proche du seuil, sauf en cas de refus déjà déclenché.

4. Contraintes réglementaires

- Les clients de moins de 18 ans sont systématiquement refusés, conformément à la réglementation.

La fonction parcourt chaque individu et applique ces règles dans cet ordre, renvoyant une liste des décisions finales qui peuvent être `ACCEPTÉ` , `REFUSE` , ou `REJET TEMPORAIRE` .

Cette approche permet d'intégrer à la fois le scoring probabiliste et des critères métier et réglementaires complexes dans la prise de décision.

```
In [25]: def decision_finale(df, proba_column='y_prob_platt'):
    decisions = []

    for _, row in df.iterrows():
        proba = row[proba_column]

        ### 1. Seuils adaptatifs par segment
        if row['Revenus'] < 1500:
            seuil = 0.55
        elif row['Revenus'] < 3000:
            seuil = 0.35
        else:
            seuil = 0.20

        ### 2. Zone d'incertitude
        zone_basse = seuil - 0.05
        zone_haute = seuil + 0.05

        if proba < zone_basse:
            decision = 'ACCEPTTE'
        elif proba > zone_haute:
            decision = 'REFUSE'
        else:
            decision = 'REJET TEMPORAIRE'

        ### 3. Règles hiérarchiques
        if row['Score_Bureau'] < 400:
            decision = 'REFUSE'

        if row['Anciennete_Emploi'] > 10 and row['Revenus'] > 2500 and decision
            decision = 'ACCEPTTE'

        ### 4. Contraintes réglementaires
        if row['Age'] < 18:
            decision = 'REFUSE'

        decisions.append(decision)

    return decisions
```

Application de la fonction de décision finale au jeu de test

Cette ligne applique la fonction `decision_finale` sur le DataFrame `df_test` pour générer une nouvelle colonne `decision`.

- Chaque ligne reçoit une décision finale parmi : `ACCEPTTE`, `REFUSE`, ou `REJET TEMPORAIRE`.
- Ces décisions intègrent les seuils adaptatifs, règles métier, zones d'incertitude et contraintes réglementaires définies précédemment.

Cela permet d'analyser et d'exploiter directement les résultats de la stratégie de décision avancée sur les données de test.

```
In [26]: df_test['decision'] = decision_finale(df_test)
```

```
In [27]: print(df_test['decision'].value_counts())  
         print(df_test['decision'].value_counts(normalize=True))
```

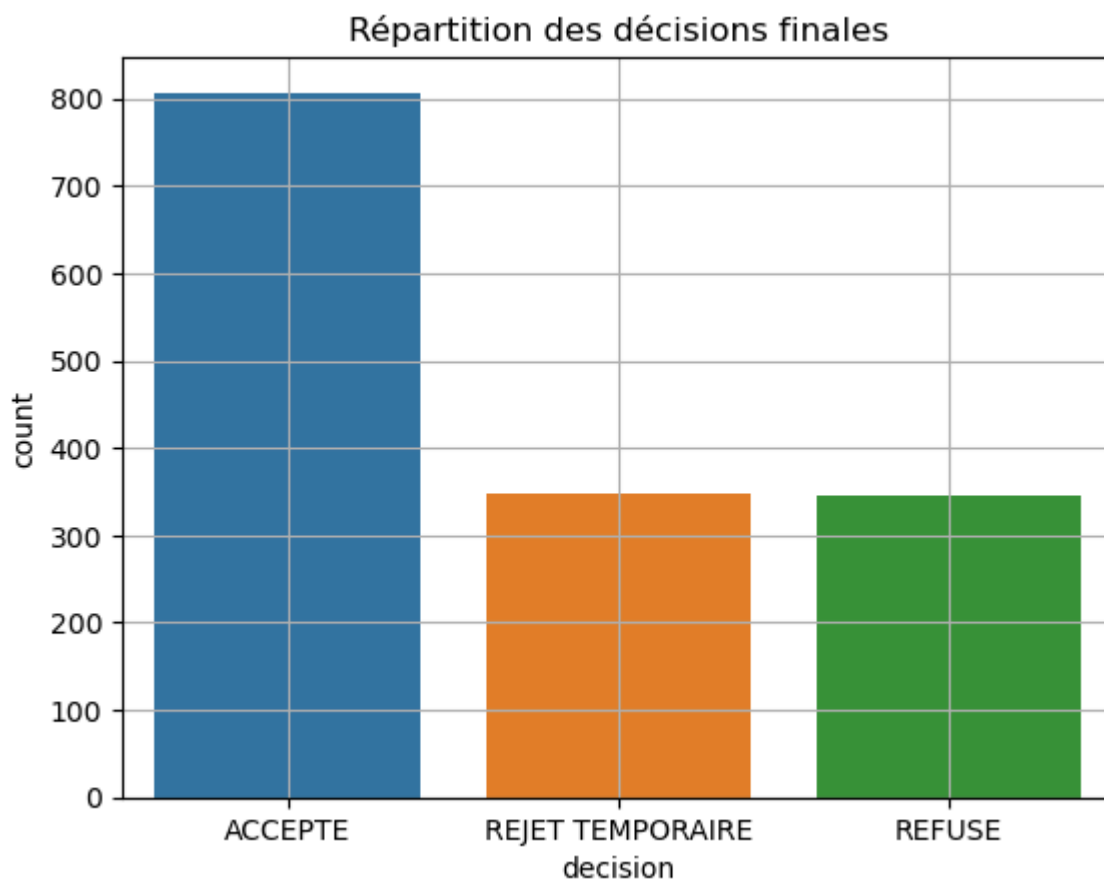
```
ACCEPTÉ           807  
REJET TEMPORAIRE  347  
REFUSE            346  
Name: decision, dtype: int64  
ACCEPTÉ           0.538000  
REJET TEMPORAIRE  0.231333  
REFUSE            0.230667  
Name: decision, dtype: float64
```

Visualisation de la répartition des décisions finales

Ce graphique affiche la **distribution des décisions** prises par la fonction de décision finale sur le jeu de test.

- Le diagramme en barres (`countplot`) montre le nombre d'individus acceptés, refusés, ou temporairement rejetés.
- L'ordre des catégories est forcé pour une meilleure lisibilité :
ACCEPTÉ , REJET TEMPORAIRE , puis REFUSE .
- Cette visualisation permet d'évaluer l'impact global de la stratégie de décision sur la population testée.

```
In [28]: import seaborn as sns  
sns.countplot(data=df_test, x='decision', order=['ACCEPTÉ', 'REJET TEMPORAIRE',  
plt.title("Répartition des décisions finales")  
plt.grid(True)  
plt.show()
```



évaluation du coût par stratégie

Fonction de calcul du coût total associé aux décisions

Cette fonction `calcul_cout_total` calcule le **coût financier total** des erreurs du modèle, en tenant compte de :

- **Faux positifs (FP)** : clients incorrectement classés comme mauvais payeurs, générant un coût élevé (`cout_fp` , par défaut 5000).
- **Faux négatifs (FN)** : clients mauvais payeurs non détectés (acceptés à tort), avec un coût moindre (`cout_fn` , par défaut 500).

La fonction utilise la matrice de confusion pour extraire les nombres de FP et FN, puis calcule la somme pondérée de ces erreurs.

Cette métrique économique est essentielle pour évaluer l'impact réel du modèle sur l'entreprise.

```
In [29]: def calcul_cout_total(y_true, y_pred, cout_fp=5000, cout_fn=500):
          tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
          return fp * cout_fp + fn * cout_fn
```

Ce bloc crée un DataFrame `df_test` combinant :

- Les variables explicatives du jeu de test (`X_test`).
- La vraie cible `Defaut_12M` pour référence et évaluation.
- Les probabilités prédites recalibrées (`y_prob_platt`).

Ce DataFrame complet permet d'analyser les décisions du modèle en lien avec les vraies valeurs et les scores prédits.

```
In [30]: # Recréer un DataFrame complet de test avec Les bonnes colonnes
df_test = X_test.copy()
df_test['Defaut_12M'] = y_test # Ajoute la vraie valeur de la cible
df_test['y_prob_platt'] = y_prob_platt # Ajoute la proba prédite
```

Attribution des décisions finales au DataFrame test

Cette ligne applique la fonction `decision_finale` sur `df_test` pour déterminer la décision finale (`ACCEPTÉ` , `REFUSE` , `REJET TEMPORAIRE`) pour chaque observation, en combinant les probabilités prédites et les règles métier.

Cela enrichit le DataFrame pour permettre une analyse complète des décisions et de leur impact.

```
In [31]: df_test['decision'] = decision_finale(df_test)
```

Comparaison des coûts économiques selon différentes stratégies de décision

Ce bloc calcule le coût total des erreurs pour trois approches distinctes :

1. Seuil fixe à 0.5

- Prédications binaires basées sur un seuil classique à 0.5.
- Calcul du coût total des faux positifs et faux négatifs selon la fonction

`calcul_cout_total`.

2. Seuil optimisé basé sur le F1-score

- Utilisation du seuil `optimal_threshold_f1` trouvé précédemment qui maximise le F1-score.
- Évaluation du coût total à ce seuil optimisé.

3. Stratégie avancée intégrant décisions adaptatives et zone d'incertitude

- Conversion des décisions finales en labels binaires : `REFUSE` = 1 (risque détecté), `ACCEPTE` = 0, et `REJET TEMPORAIRE` = -1 (exclusion temporaire).
- Les cas avec `REJET TEMPORAIRE` sont exclus de cette évaluation car la décision est différée.
- Calcul du coût total uniquement sur les cas validés (acceptés ou refusés).

Cette comparaison permet de quantifier l'amélioration potentielle du modèle et des règles métier sur la base du coût financier pour l'entreprise.

```
In [32]: #Comparaison entre 3 stratégies :

# 1. Seuil fixe 0.5
y_pred_simple = (y_prob_platt >= 0.5).astype(int)
cout_simple = calcul_cout_total(y_test, y_pred_simple)

# 2. Seuil optimisé F1
y_pred_f1 = (y_prob_platt >= optimal_threshold_f1).astype(int)
cout_f1 = calcul_cout_total(y_test, y_pred_f1)

# 3. Stratégie avancée
y_pred_strategique = df_test['decision'].map({'REFUSE': 1, 'ACCEPTE': 0, 'REJET
df_valid = df_test[y_pred_strategique != -1] # on exclut les cas rejetés

cout_strategique = calcul_cout_total(df_valid['Defaut_12M'], y_pred_strategique[

In [33]: print(f"Coût stratégie simple : {cout_simple} €")
print(f"Coût seuil optimisé F1 : {cout_f1} €")
print(f"Coût stratégie avancée : {cout_strategique} €")
```

Coût stratégie simple : 289500 €
 Coût seuil optimisé F1 : 1340500 €
 Coût stratégie avancée : 1101500 €

Analyse de sensibilité aux variations de marché Exemple : que se passe-t-il si le taux de défaut double (crise) ou baisse fortement (boom) ?

```
def simulate_y_with_default_rate(y_true, new_default_rate=0.3): n = len(y_true) n_default = int(new_default_rate * n) y_sim =
np.zeros(n) y_sim[:n_default] = 1 np.random.shuffle(y_sim) return y_sim.astype(int) # Exemple : passer à 30% de défaut
y_test_stress = simulate_y_with_default_rate(y_test, new_default_rate=0.3) cout_stress = calcul_cout_total(y_test_stress,
y_pred_strategique[y_pred_strategique != -1])
```

Analyse du coût total en fonction du seuil de décision

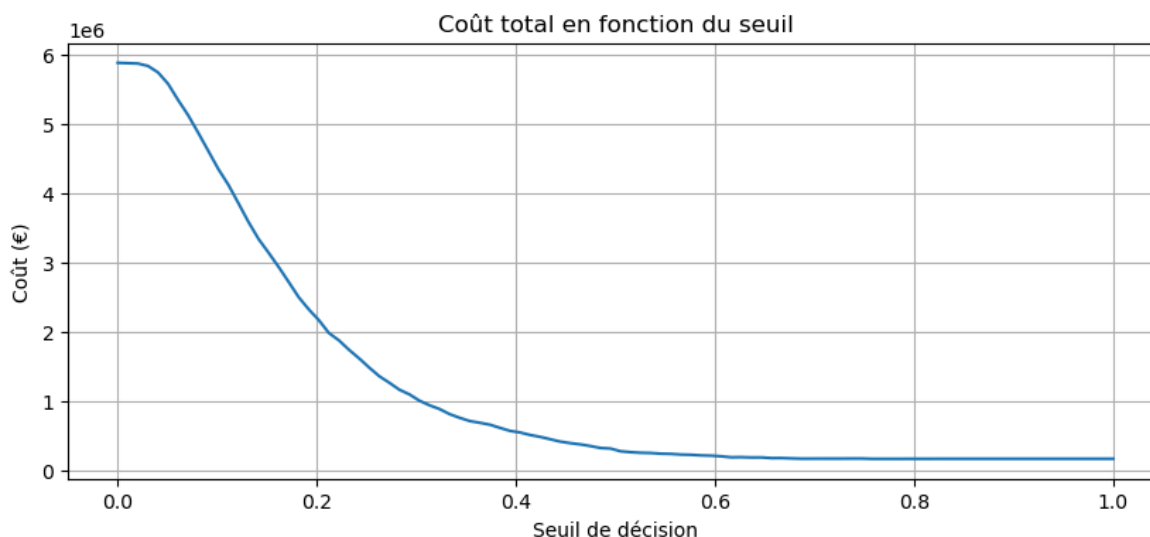
Ce bloc évalue comment le coût total (faux positifs et faux négatifs pondérés économiquement) évolue en faisant varier le seuil de décision de 0 à 1.

- Pour chaque seuil `t`, on calcule les prédictions binaires et le coût total associé grâce à la fonction `calcul_cout_total`.
- La courbe trace ce coût en fonction du seuil, permettant d'identifier visuellement le seuil minimisant le coût économique.
- Cette analyse est cruciale pour choisir un seuil non seulement performant en métriques classiques, mais aussi optimisé pour l'impact financier réel.

```
In [35]: import numpy as np
thresholds = np.linspace(0, 1, 100)
costs = []

for t in thresholds:
    y_pred = (y_prob_platt >= t).astype(int)
    cost = calcul_cout_total(y_test, y_pred)
    costs.append(cost)

plt.figure(figsize=(10, 4))
plt.plot(thresholds, costs)
plt.title("Coût total en fonction du seuil")
plt.xlabel("Seuil de décision")
plt.ylabel("Coût (€)")
plt.grid(True)
plt.show()
```



In []:

In []: